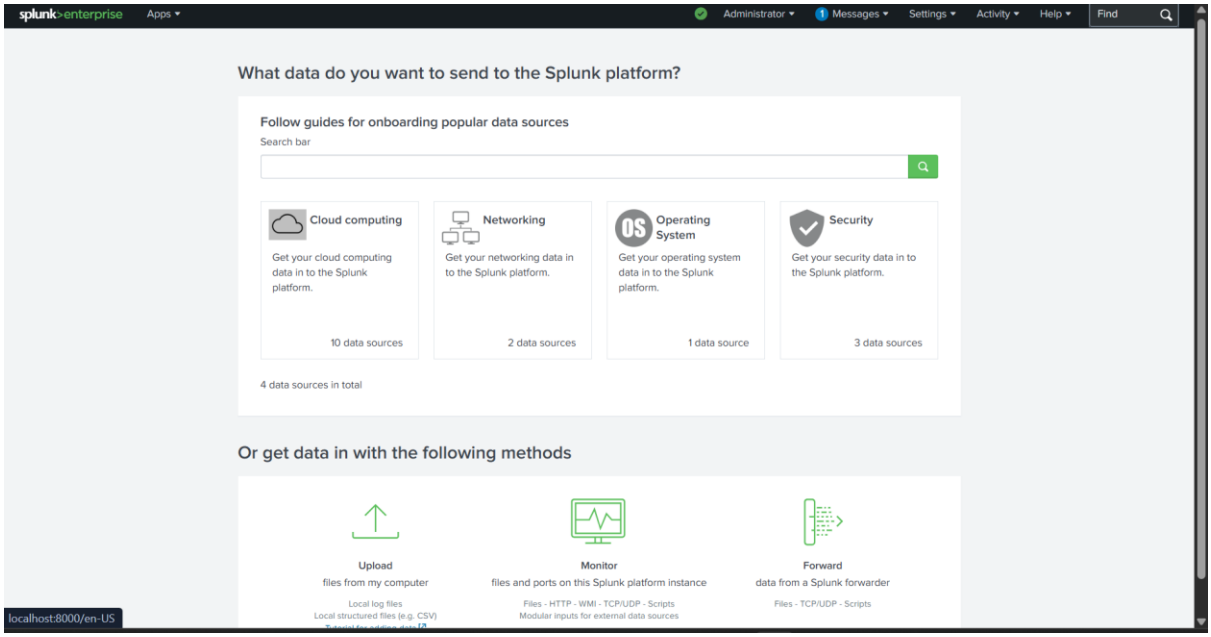
# Lab 2 - Splunk Dashboard for Web Traffic Logs

## Objective

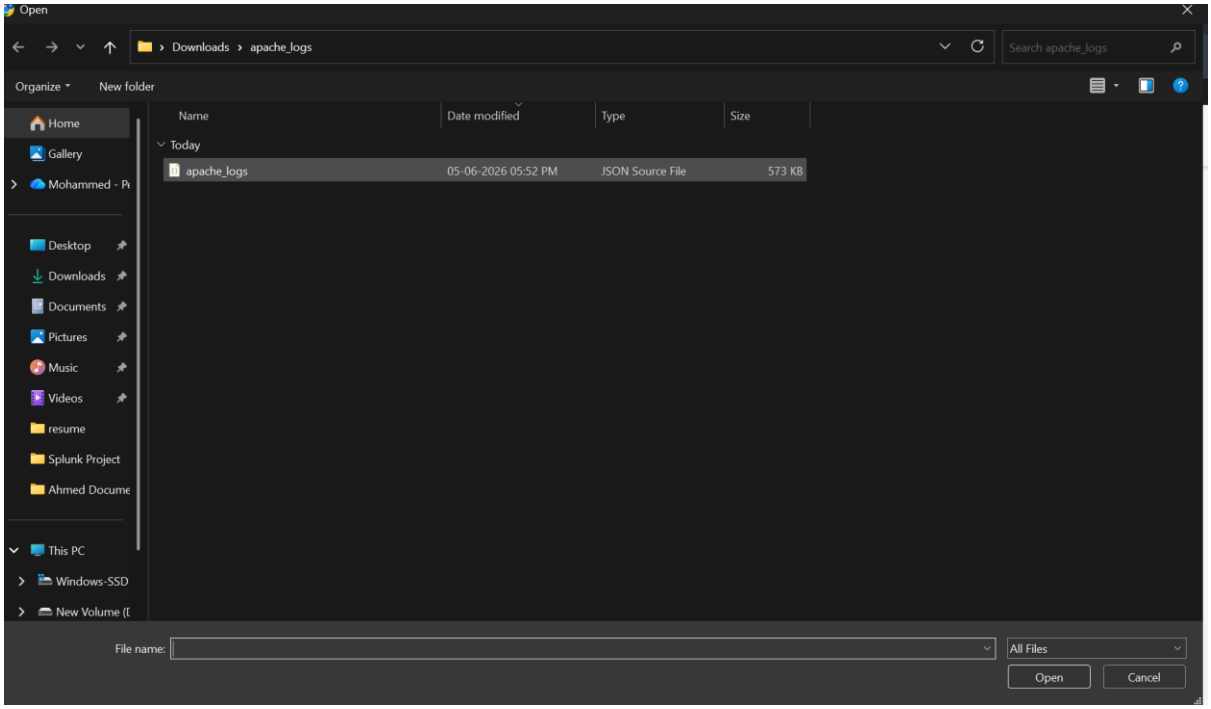
To create a Splunk dashboard that visualizes web server activity, response statistics, user access patterns, and geographic traffic distribution for effective monitoring and analysis of Apache web logs.

---

At first, Log in to the Splunk Web interface.
Navigate to Settings → Add Data or click Add Data from the Splunk home page.



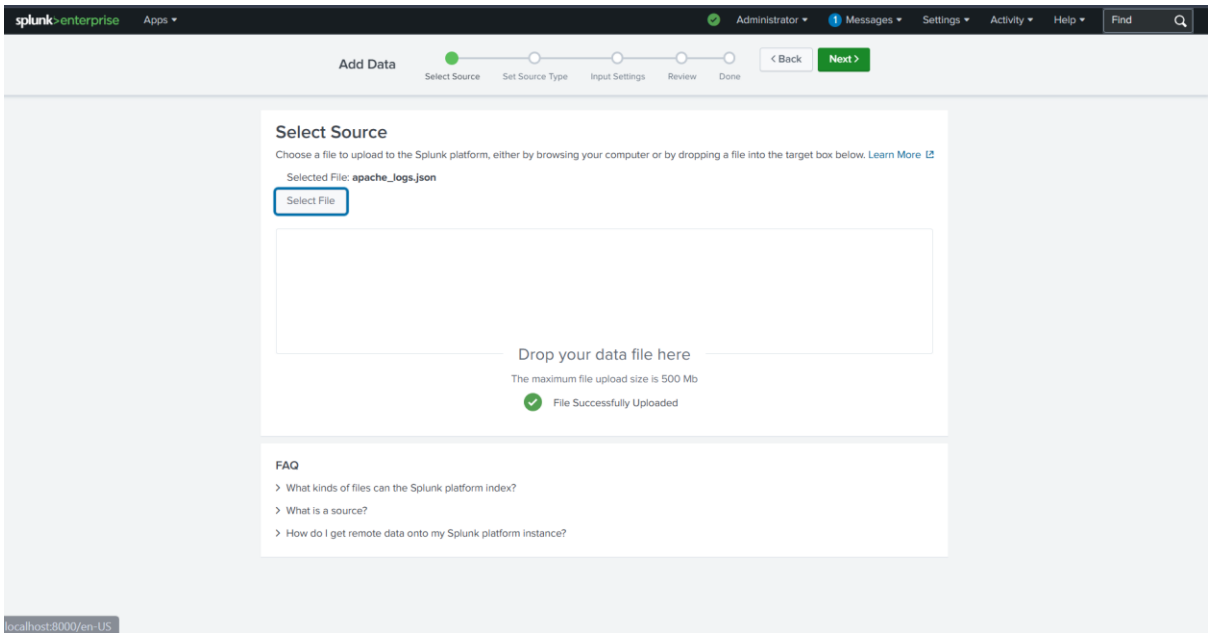

Click on >> Upload




Select the logs file [Link]

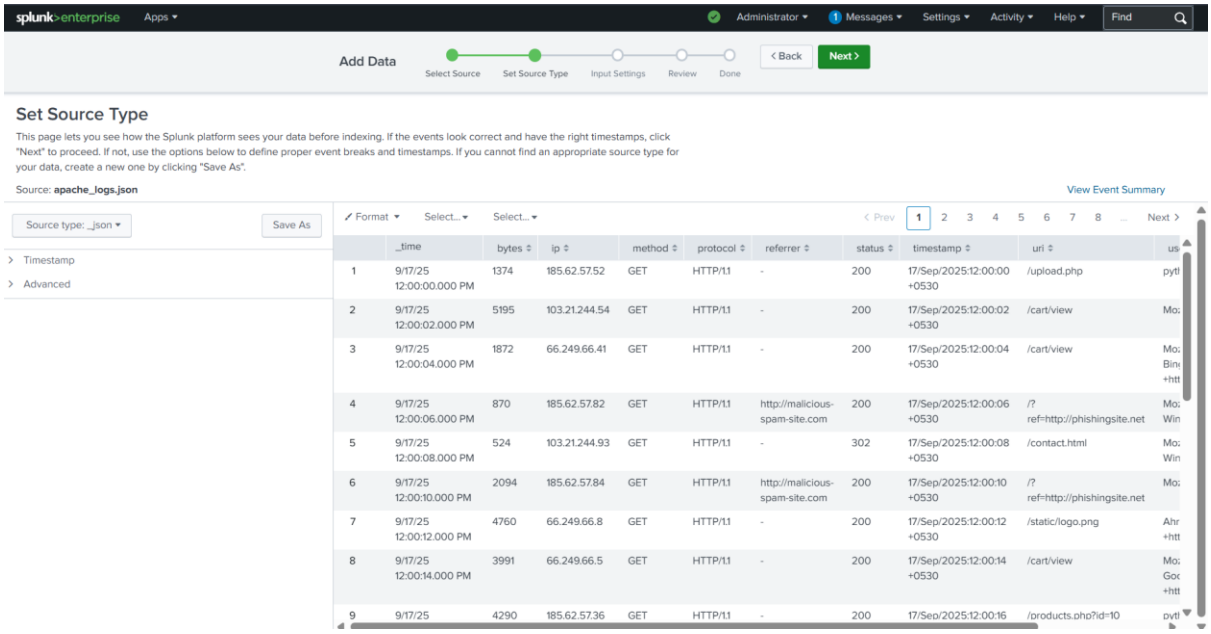
---

The file is uploaded into Splunk




Click on >> Next

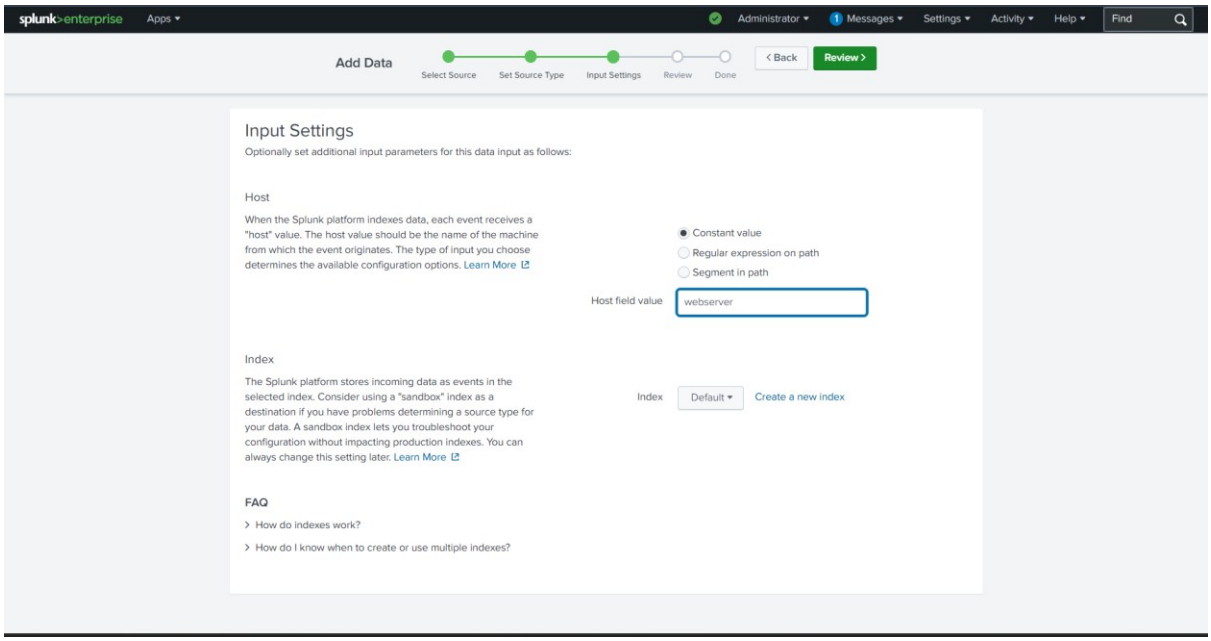
This are the logs




Click on >> Next

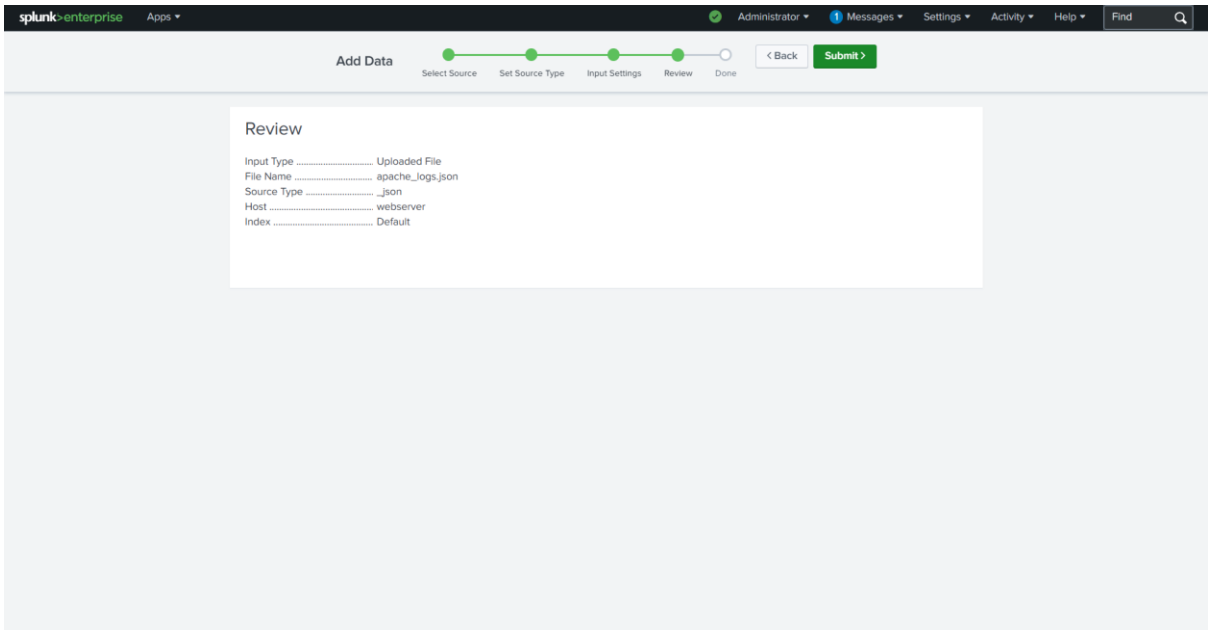
---

Let us keep our host name as webserver




Click on >> Review

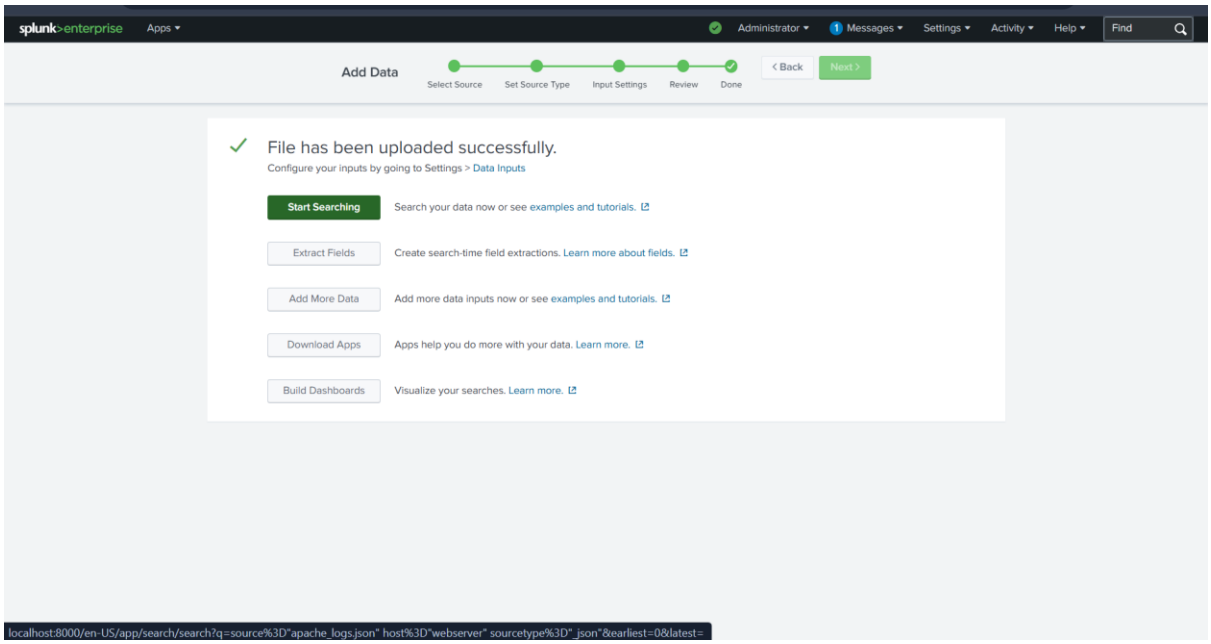
This is an overview of our uploaded file




Click on >> Submit

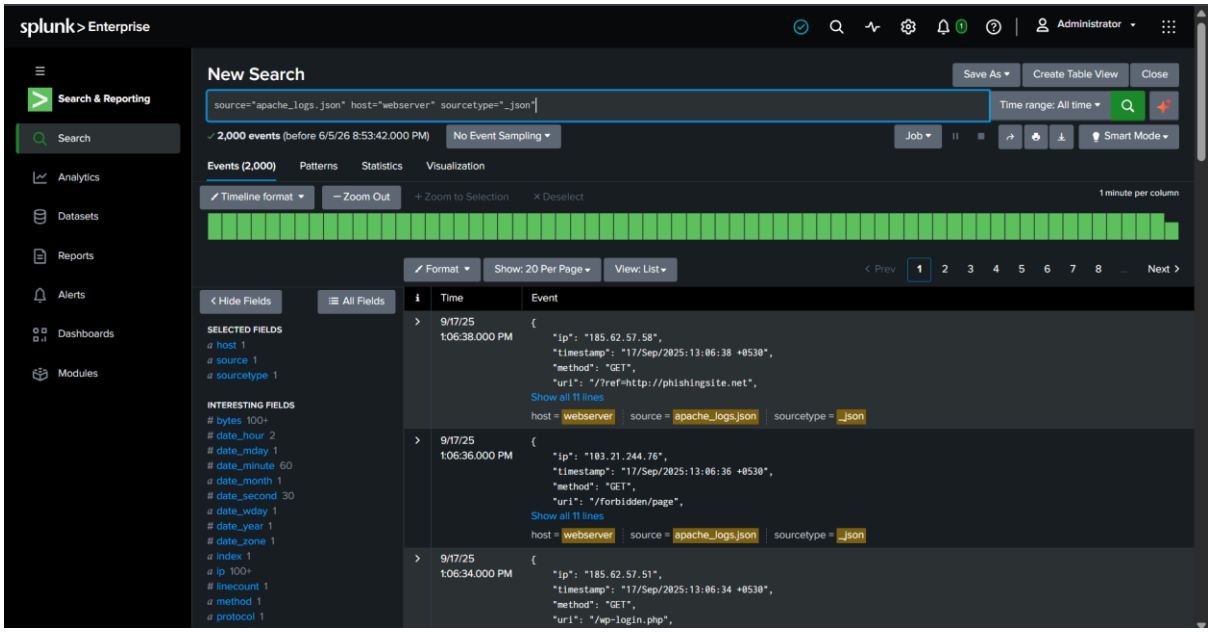
---

Now that our logs are ready let us start Searching




Click on >> Start Searching

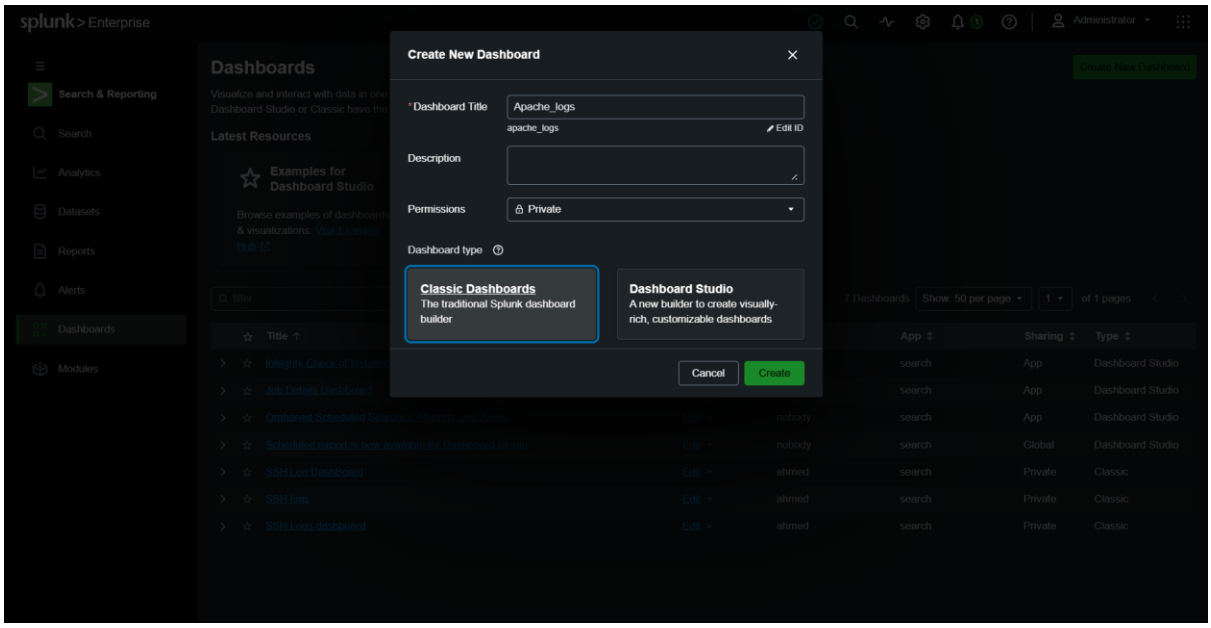
This is the search menu where we can get our desired output by querying




Click on >> Dashboard

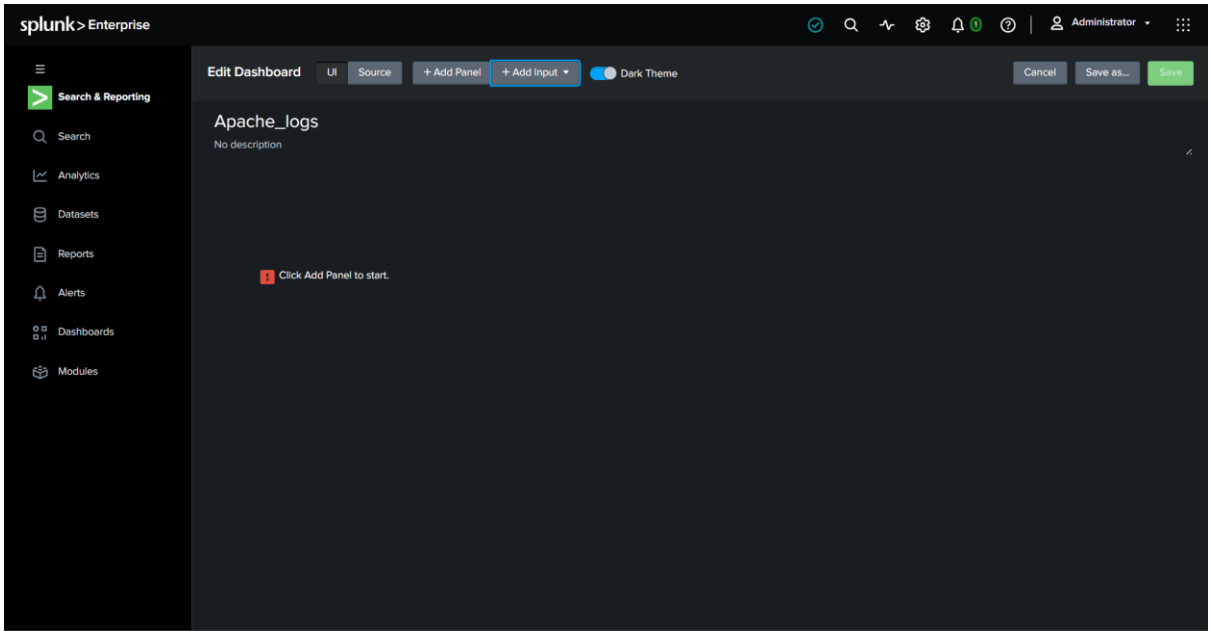
---

Set a Dashboard tile and select the dashboard type as Classic



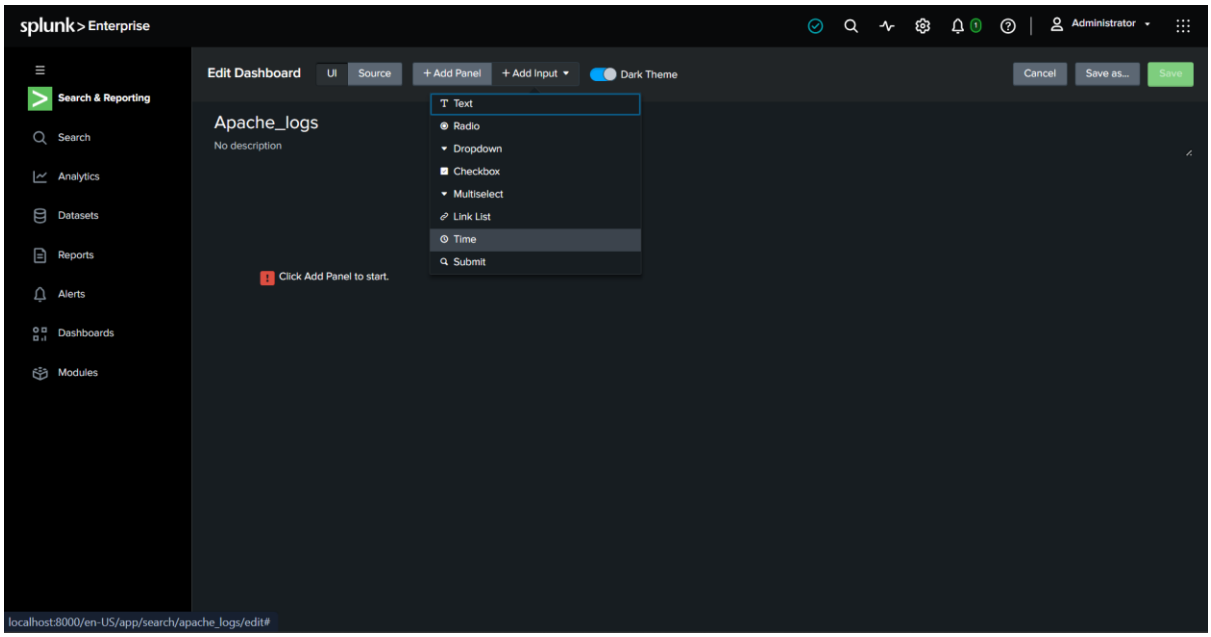

Click on >> Create

Now is this blank dashboard, let us add time and submit button here



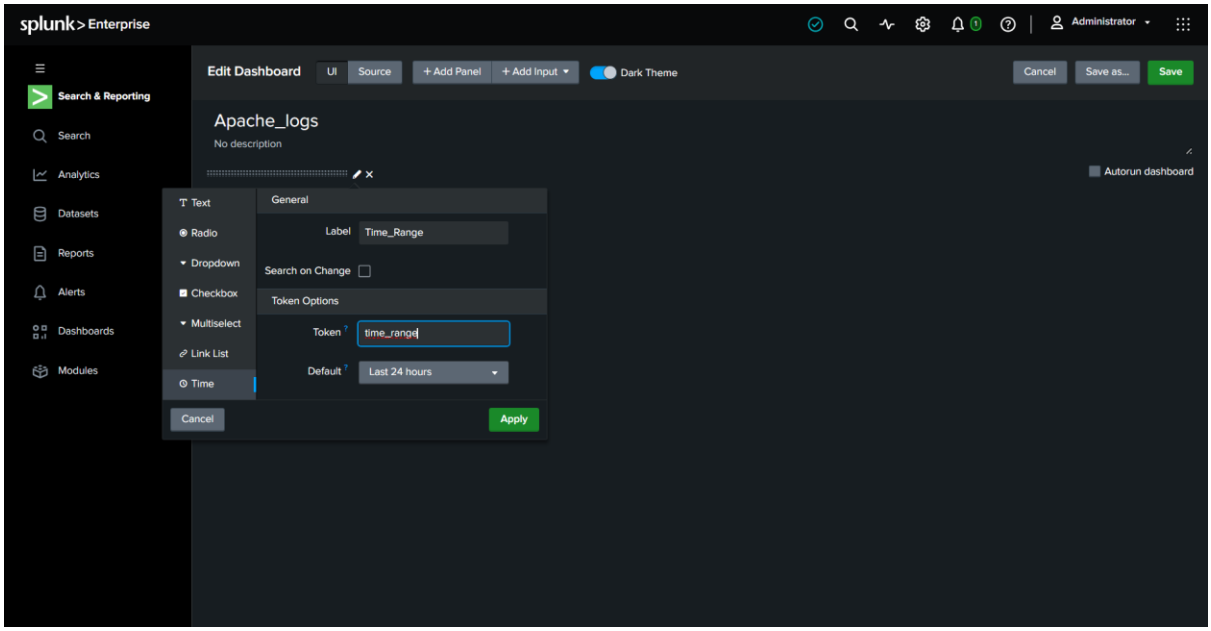

Click on >> Add Input

---




Click on >> Add Input >> Time

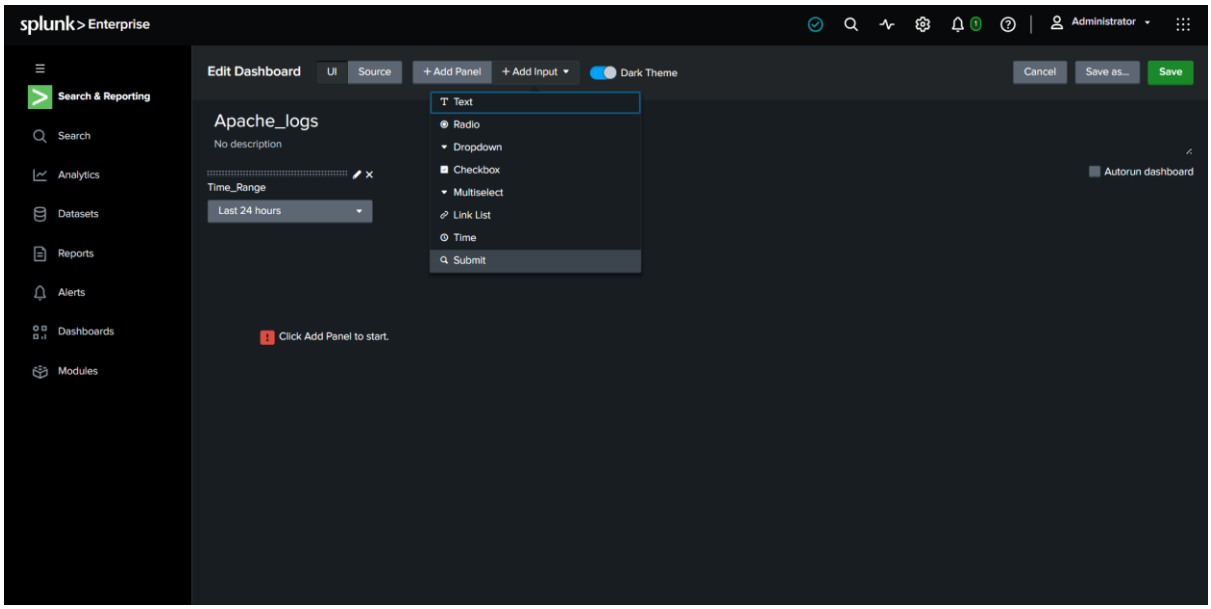
Click on pencil icon and set time range




Click on >> Apply

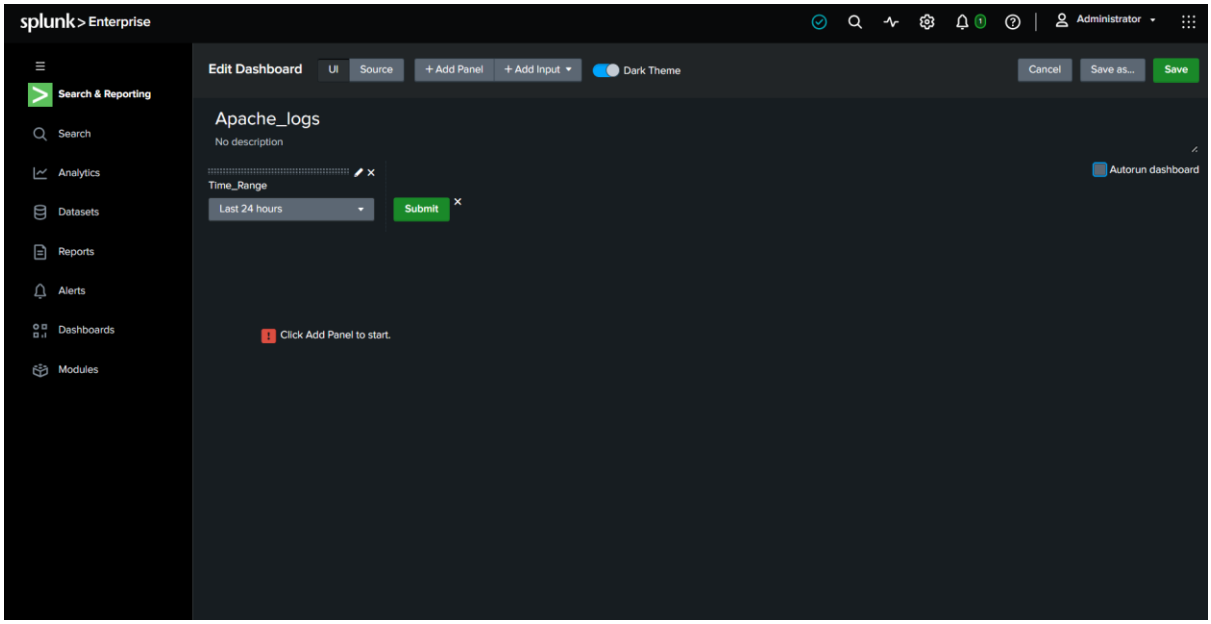
---

As we can see time range is added in the dashboard, now let us add submit button




Click on >> Add Input >> Submit

Now let us do some querying



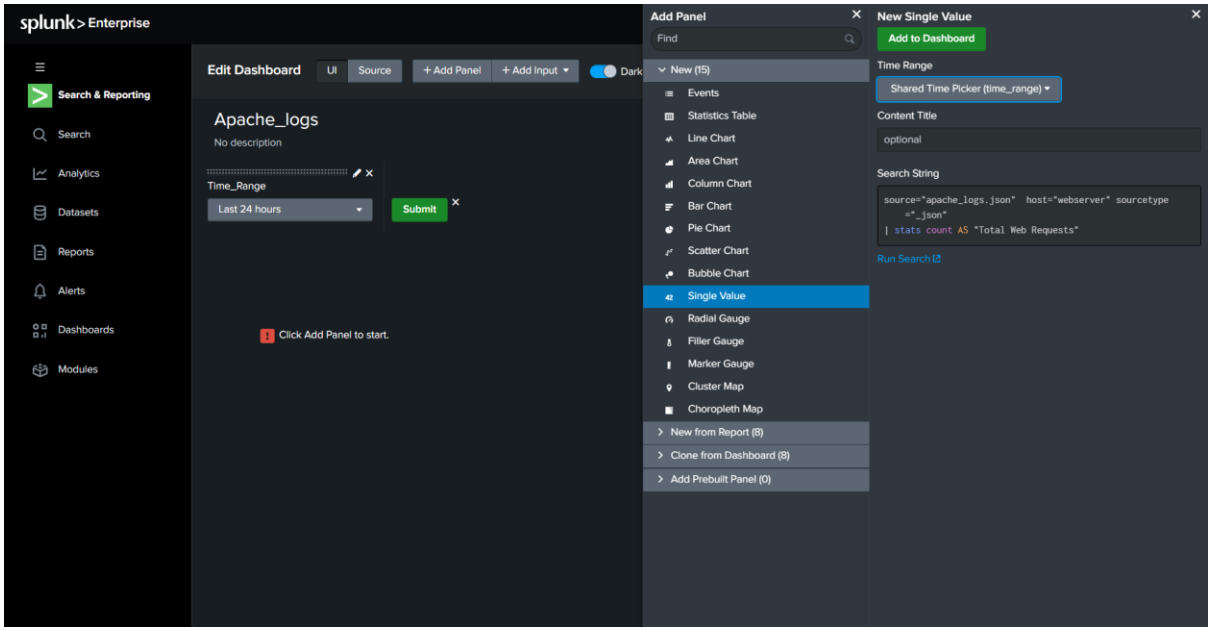

**1. Total Web Requests**

- Click on Add Panel
- Under New, choose Single Value
- Use Shared Time Picker **time_range**
- Set Content Title to "Total Web Requests"
- Enter the Search String as below

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| stats count AS "Total Web Requests"
```

**Explanation:**

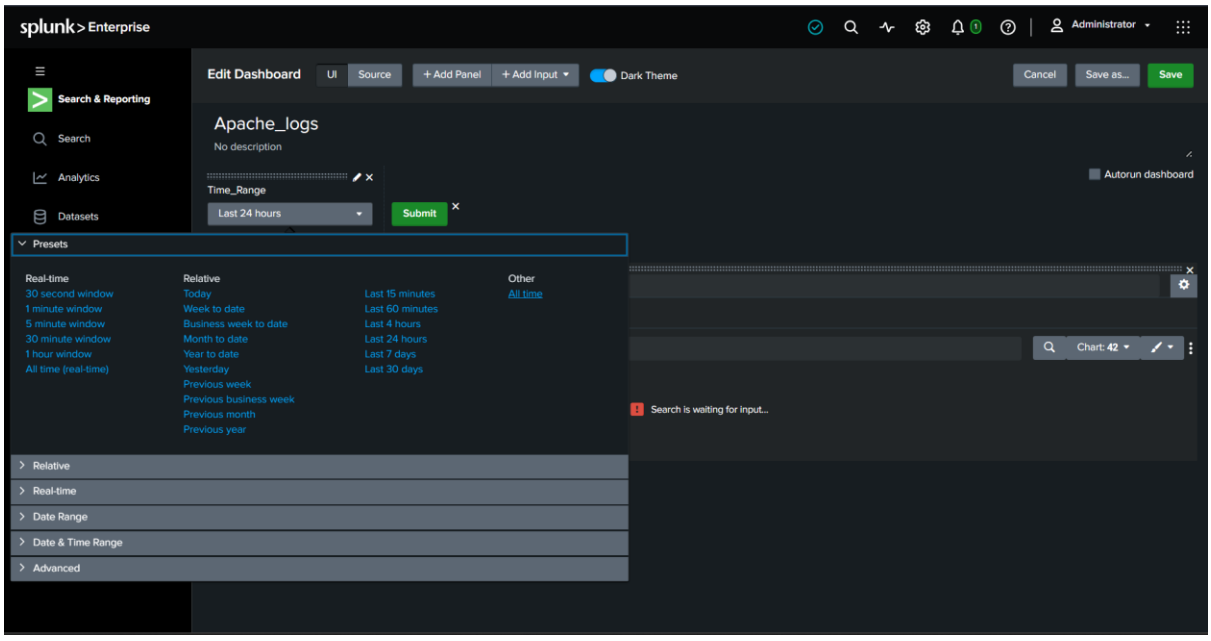
- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Counts the total number of events (web requests) in the selected time range using the stats command.
- Renames the output field to **"Total Web Requests"** for better dashboard readability.




Click on >> Add to Dashboard

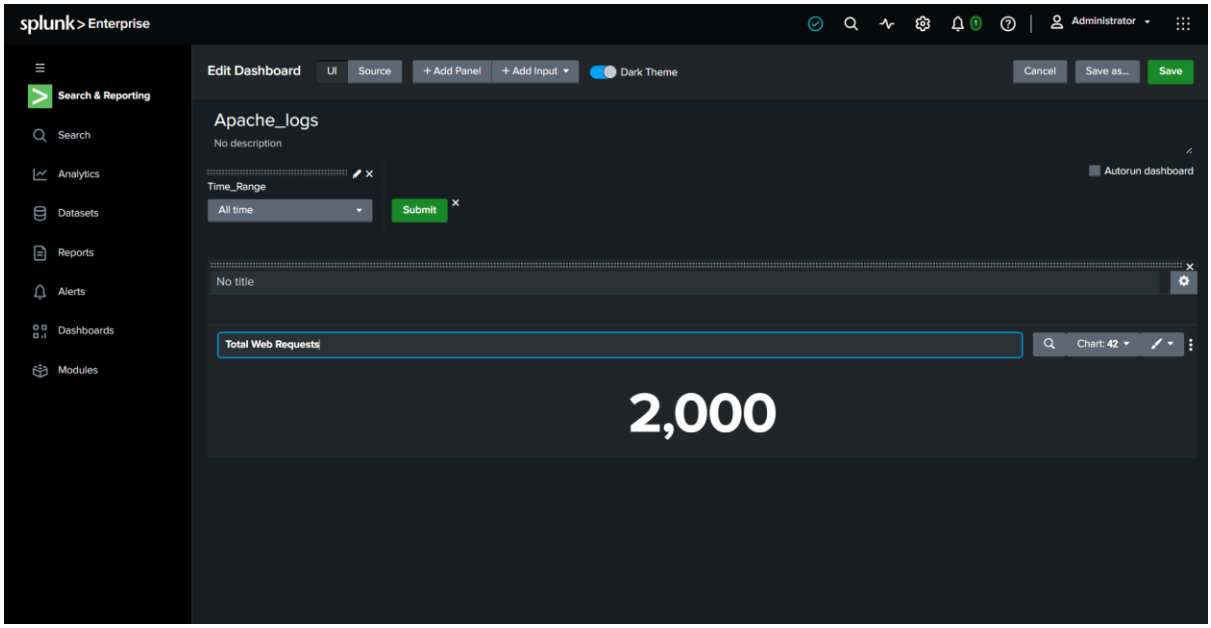
---

As we can see no logs are generated, so we need to set the time rage to “All time”




Click on >> Time_Range >> All Time

We can see the logs now!




---

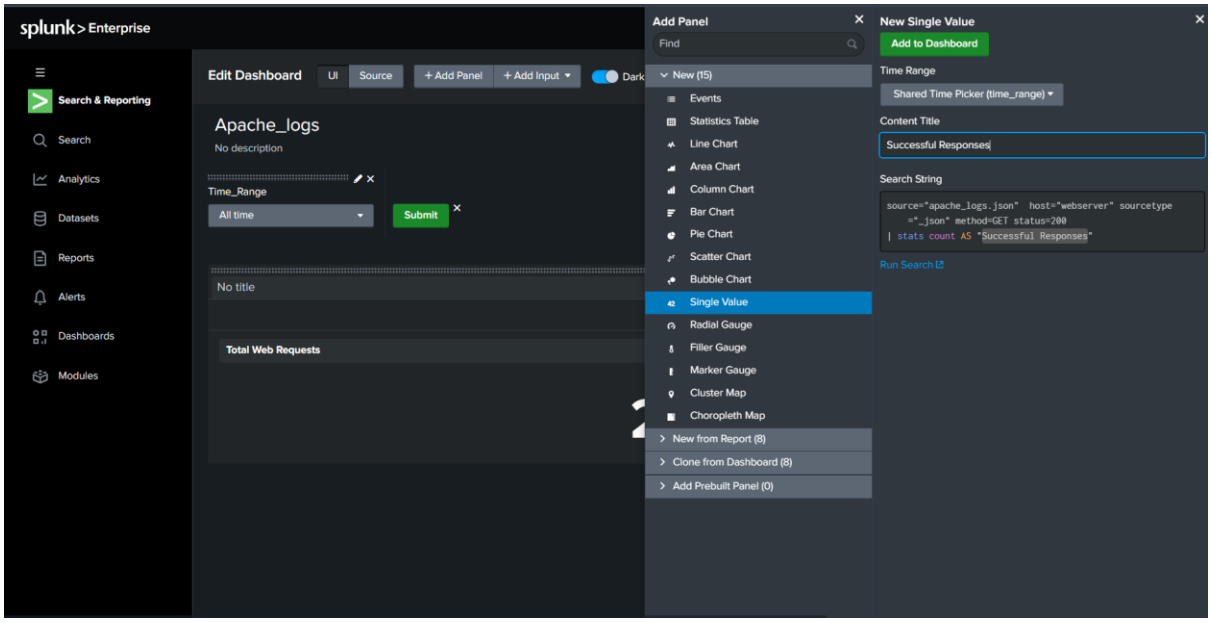
**2. Successful Response**

- Click on Add Panel
- Under New, choose Single Value
- Use Shared Time Picker **time_range**
- Set Content Title to "Successful Response"
- Enter the Search String as below

```
source="apache_logs.json"  host="webserver" sourcetype="_json"
method=GET status=200
| stats count AS "Successful Responses"
```

**Explanation:**

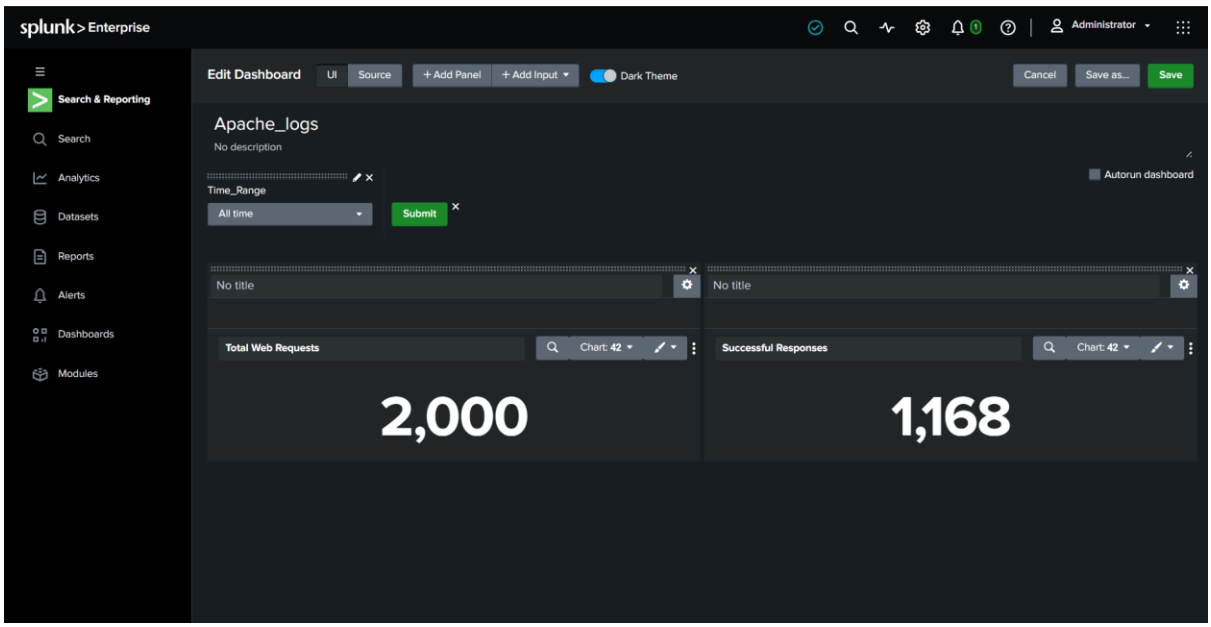
- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Selects only **GET** requests.
- Filters for HTTP **status code 200**, indicating successful requests.
- Counts the total number of successful responses using the stats command.
- Renames the output field to **"Successful Responses"** for clear dashboard visualization.




Click on >> Add to Dashboard

---

We can see logs being displayed




---

### 3. Client Errors and Server Error

- Click on Add Panel
- Under New, choose Single Value
- Use Shared Time Picker time_range
- Set Content Title to "Client Errors"

- Enter the Search String as below:

```
source="apache_logs.json"  host="webserver" sourcetype="_json"
| where status>=400 and status<500
| stats count AS "Client Errors"
```

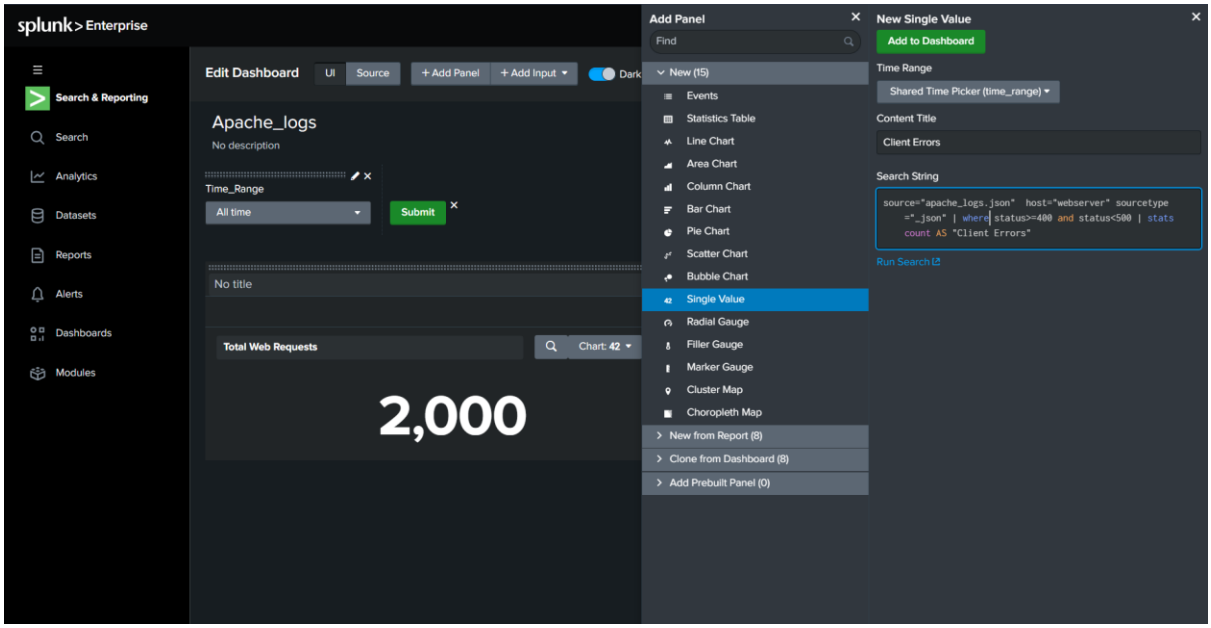
**Explanation:**

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The where command filters events with HTTP status codes between **400 and 499**.
- These status codes represent **client-side errors**, such as:
    - **400** – Bad Request
    - **401** – Unauthorized
    - **403** – Forbidden
    - **404** – Not Found
- The stats count command counts the total number of matching error events.
- Renames the output field to **"Client Errors"** for better readability in the dashboard.

```
source="apache_logs.json" host="webserver" sourcetype="_json"
| where status<500
| stats count AS "Server Errors"
```

**Difference from the Client Error Query:**

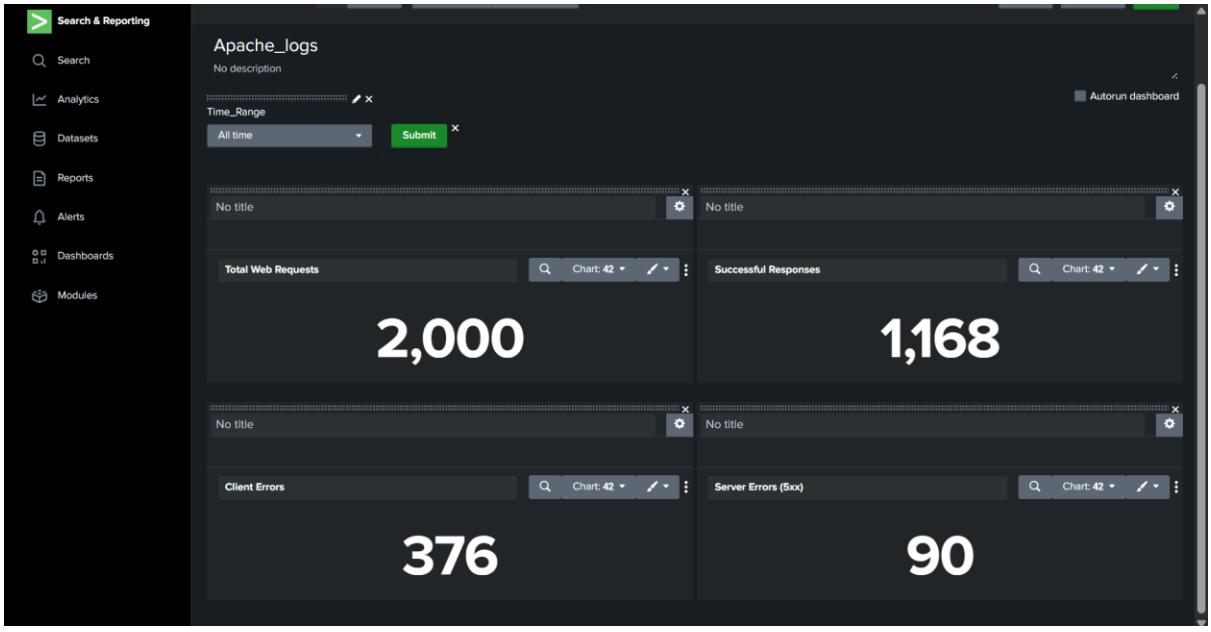
- The where condition is changed from status>=400 and status<500 to status>=500.
- This filters for **5xx HTTP status codes**, which indicate **server-side failures** rather than client-side issues.
- Examples include:
    - **500** – Internal Server Error
    - **502** – Bad Gateway
    - **503** – Service Unavailable
    - **504** – Gateway Timeout
- The output field is renamed to **"Server Errors"**.




Click on >> Add to Dashboard

---

We can see Client error and Serve error being displayed




---

### 4. Top Requested URIs

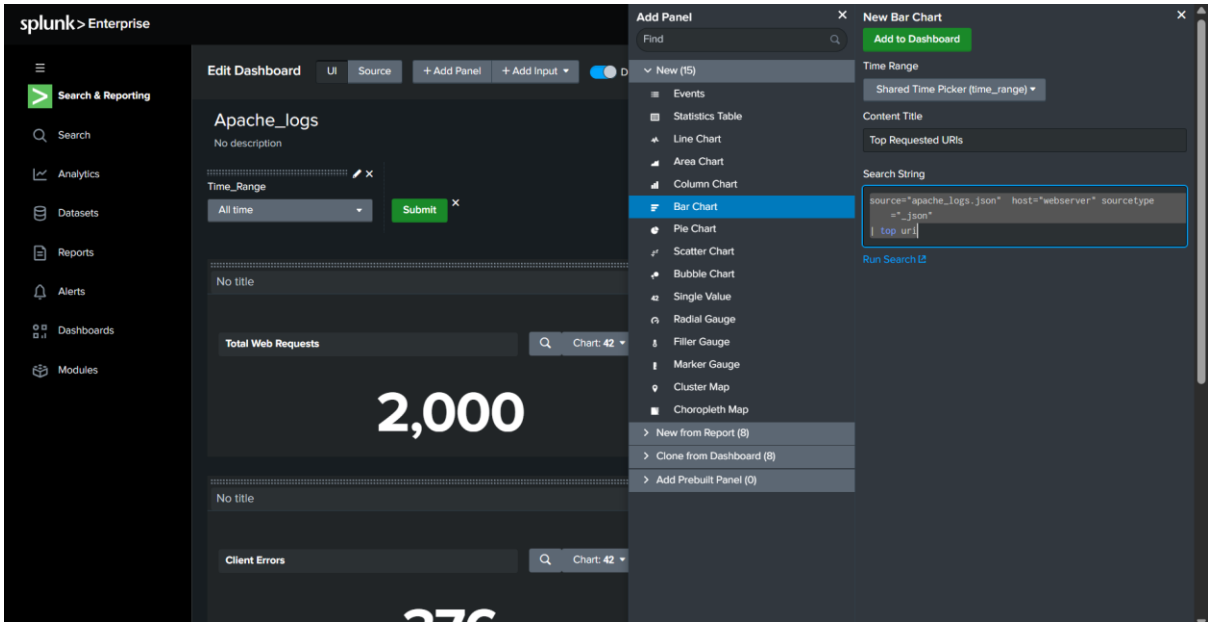
- Click on Add Panel
- Under New, choose Bar Chart
- Use Shared Time Picker **time_range**
- Set Content Title to "Top Requested URIs"

- Enter the Search String as below

```
source="apache_logs.json"  host="webserver" sourcetype="_json"
| top uri
```

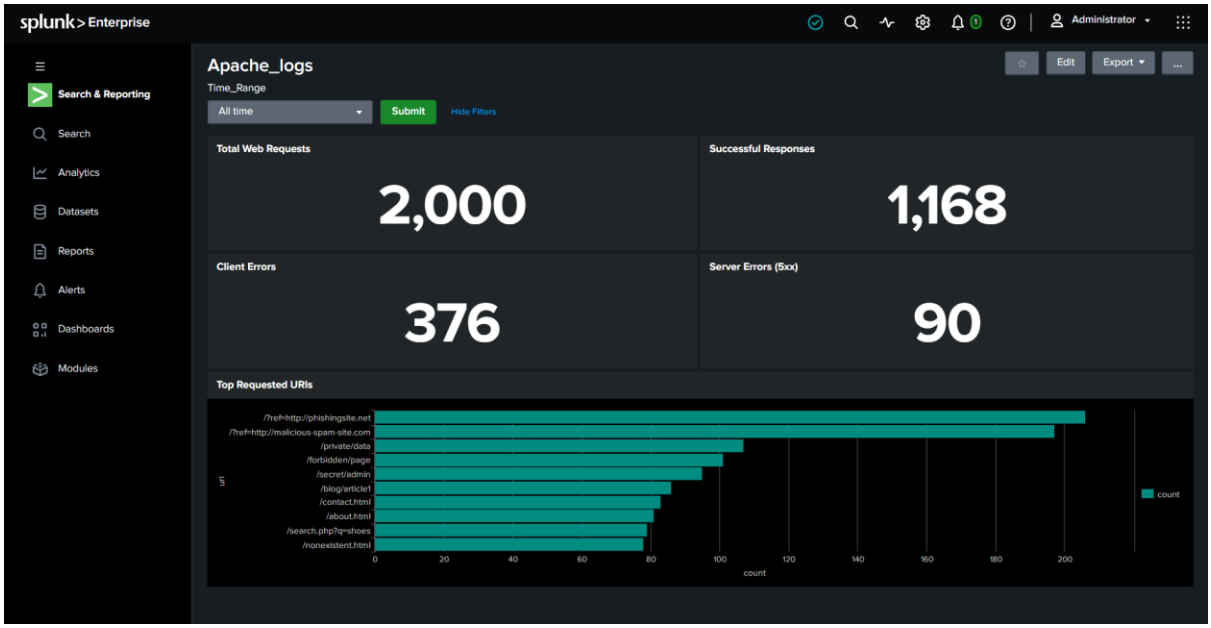
**Explanation:**

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The top command identifies the most frequently occurring values in the uri field.
- The uri field represents the requested resource or endpoint on the web server.
- Splunk automatically calculates the count and percentage of requests for each URI and displays them in descending order.




Click on >> Add to Dashboard

We can see the bar chart in our dashboard




---

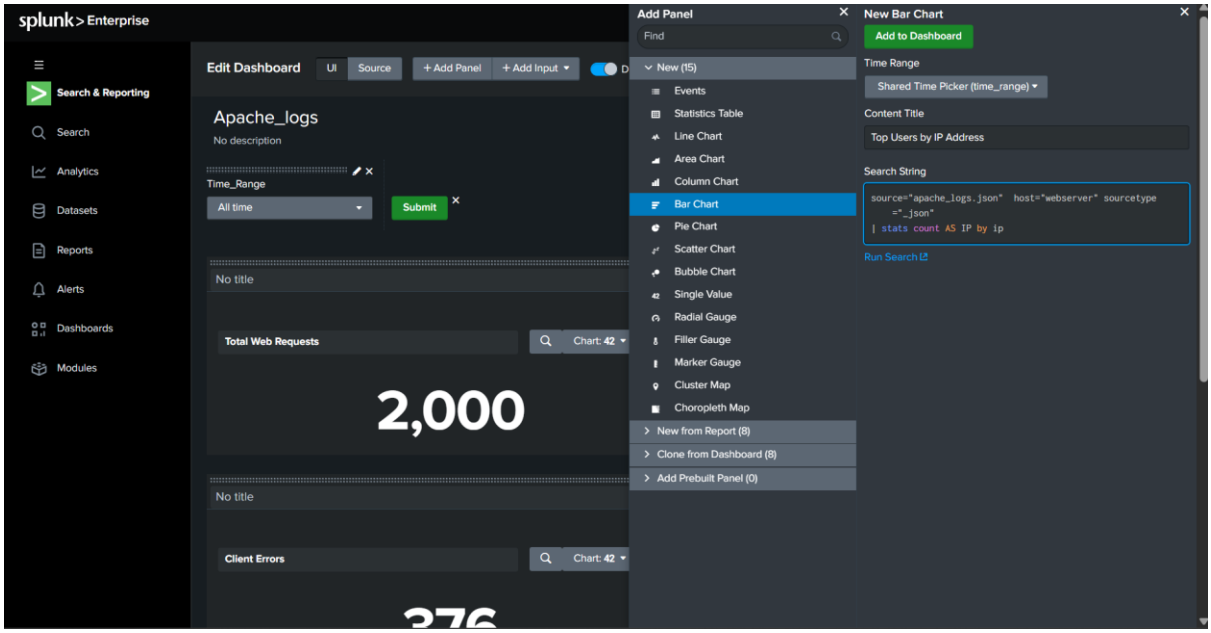
### 5. Top Users by IP Address

- Click on Add Panel
- Under New, choose Bar Chart
- Use Shared Time Picker **time_range**
- Set Content Title to "Top Users by IP Address"
- Enter the Search String as below

```
source="apache_logs.json"  host="webserver" sourcetype="_json"
| stats count AS IP by ip
```

**Explanation:**

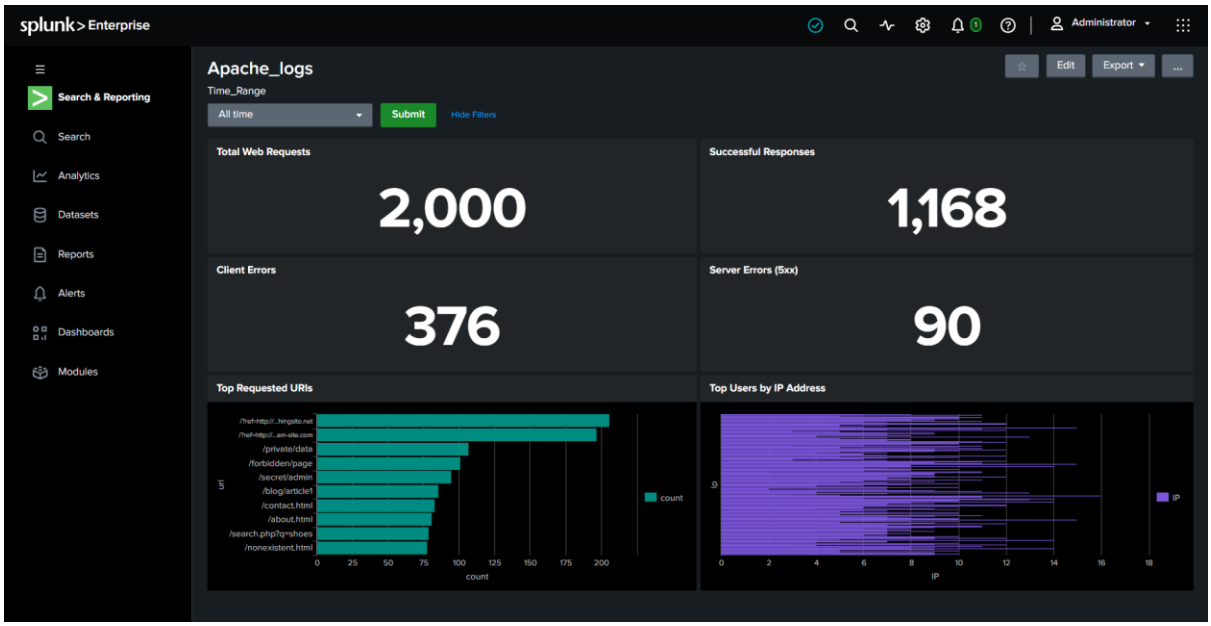
- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- The stats count by ip command groups events based on the client IP address.
- Counts the total number of requests generated by each IP address.
- Renames the count field to **IP** for display purposes.




Click on >> Add to Dashboard

---

As we can see the chart is visible on our dashboard




---

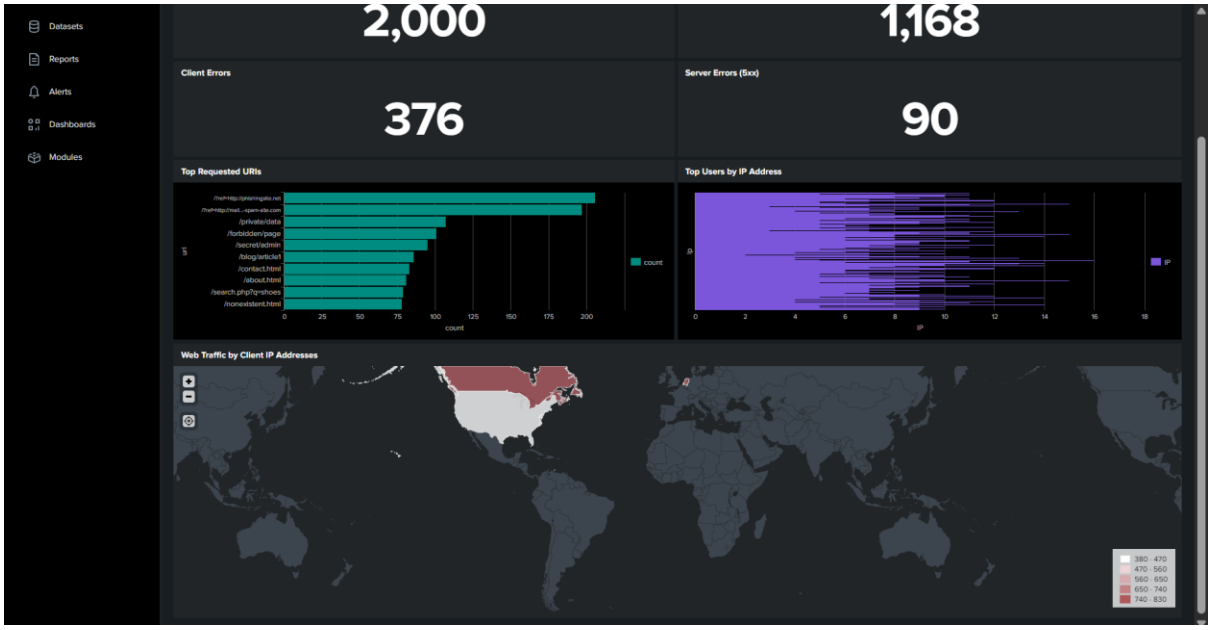
## 6. Web Traffic by Client IP Addresses

- Click on **Add Panel**
- Under **New**, choose **Choropleth Map**
- Use **Shared Time Picker time_range**
- Set **Content Title** to **Web Traffic by Client IP Addresses**
- Enter the **Search String** as below:

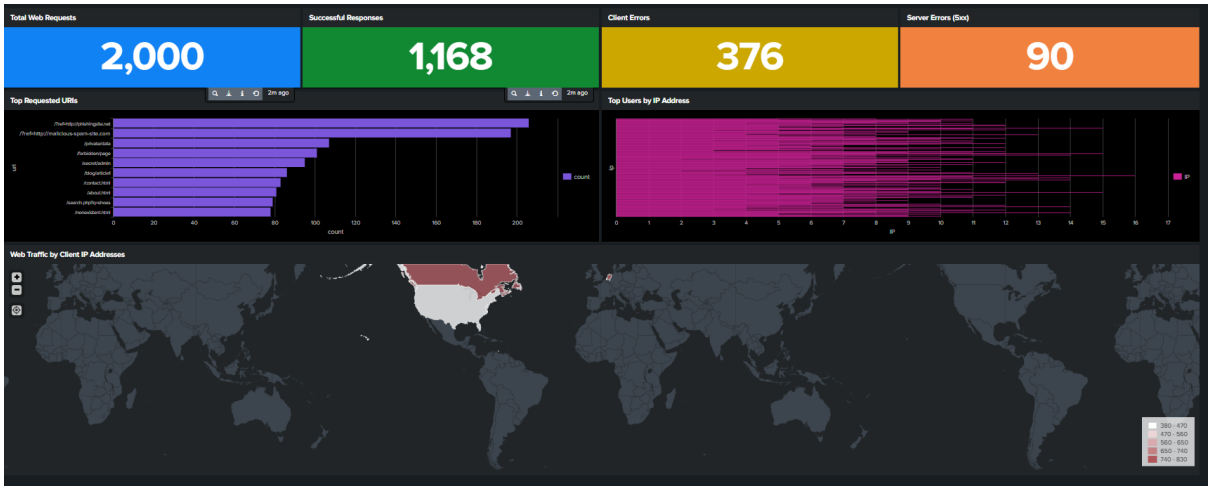
```
source="apache_logs.json" host="webserver" sourcetype="_json"
method=GET
| table ip
| iplocation ip
| stats count by Country
| geom geo_countries featureIdField="Country"
```

**Explanation:**

- Filters events from the apache_logs.json log source.
- Limits the search to logs generated by the webserver host.
- Uses the _json sourcetype to process JSON-formatted log data.
- Filters for **GET** requests, which represent users requesting web resources.
- The table ip command extracts and displays only the client IP address field.
- The iplocation command performs GeoIP lookups and enriches each IP address with geographic information, including the country.
- The stats count by Country command groups events by country and counts the number of requests originating from each location.
- The geom geo_countries featureIdField="Country" command maps the country data to geographic boundaries, enabling visualization on a choropleth map.



This is the final dashboard




---

Conclusion

In this lab, I uploaded Apache web server logs into Splunk and built a dashboard with six panels to monitor web activity. The data showed 2,000 total requests, out of which 1,168 were successful GET responses with status 200. However, 376 client-side errors and 90 server-side errors were also recorded, which in a real environment would require immediate investigation — the client errors could point to broken links or unauthorized access attempts, while the server errors may indicate backend instability. The Top Requested URIs panel revealed which pages receive the most traffic, useful for performance optimization. The IP-based panels helped identify which users are most active and where traffic is geographically coming from, with the choropleth map showing the US as a dominant traffic source. This project gave me practical experience in ingesting log data, writing SPL queries, and building visual dashboards that turn raw logs into actionable security and operational insights.